HyperLogLog: Near-Optimal Cardinality

CS-201 PROJECT | A Rigorous Analysis

What is HyperLogLog?

The Core Problem: Counting distinct items in massive data streams (Cardinality Estimation).

> Input: 1 Billion User IDs
> Goal: How many *unique* users?

The Memory Wall: Storing unique elements exactly (e.g., HashSet) requires linear memory *O(N)*. For 1 billion users, this is **~8 GB RAM**.

The Solution: HLL estimates this count with ~2% error using only **1.5 KB RAM**.

Probabilistic Data Structure

Trades precision for massive memory savings.

Real World Impact

BigQuery & Google

Used in `APPROX_COUNT_DISTINCT` functions to process petabytes of data instantly without memory overflow.

Redis

Standard data type (PFADD/PFCOUNT). Allows tracking millions of unique daily active users in a tiny memory footprint.

Cyber Security

DDoS Detection: Routers use HLL to count distinct source IPs per second. A spike indicates an attack.

The Predecessor: HLL vs LogLog

Feature	LogLog (2003)	HyperLogLog (2007)
Estimator	Geometric Mean	Harmonic Mean
Standard Error	$\approx \frac{1.30}{m}$	$\approx \frac{1.04}{m}$
Outlier Sensitivity	High	Low (Robust)

The Key Difference

LogLog uses the **Geometric Mean** to combine bucket estimates.

HyperLogLog switches to the **Harmonic Mean**.

While seemingly minor, this mathematical change significantly dampens the effect of "lucky" buckets that produce artificially high ranks.

Why HLL is Superior

The 64% Memory Argument

Since memory m is inversely proportional to the square of the error, the improvement in the error constant leads to massive savings.

$$\text{Efficiency Ratio} = \frac{1.04^2}{1.30} \approx 0.64$$

Result: HLL requires only **64% of the memory** to achieve the same accuracy as LogLog.

36%

SPACE SAVED

Algorithm Overview (Pseudocode)

```
function Add(item):
  x = Hash(item)
  j = GetBucketIndex(x)  // First p bits
  w = GetRemainingBits(x)
  rho = CountTrailingZeros(w)

  M[j] = Max(M[j], rho)

function Count():
  Z = HarmonicMean(2^M)
  E = alpha * m^2 * Z
  return CorrectBias(E)
```

The 4-Step Process

1. **Hash:** Convert data to uniform binary.
2. **Bucket:** Route to one of m registers.
3. **Measure:** Count zero-run length (ρ).
4. **Update:** Keep the MAX ρ seen.

Implementation: Init & Rank

```python
class HyperLogLog:
    def __init__(self, p=12):
        # Checklist Item 2a: Initialization
        if not (4 <= p <= 16):
            raise ValueError("p must be between 4 and 16")
        self.p = p
        self.m = 1 << p
        self.registers = [0] * self.m

        # Pre-calculate alpha constant
        if self.m == 16: self.alpha = 0.673
        elif self.m == 32: self.alpha = 0.697
        elif self.m == 64: self.alpha = 0.709
        else: self.alpha = 0.7213 / (1 + 1.079 / self.m)

    def _get_rho(self, w):
        # Helper to find rank (trailing zeros)
        if w == 0: return 32 - self.p + 1
        rho = 1
        while (w & 1) == 0:
            w >>= 1
            rho += 1
        return rho
```

Implementation: The Add Function

```
    def add(self, item):
        # Checklist Item 2b: Bit Manipulation Logic
        x = murmur3_32(str(item))
        j = x >> (32 - self.p)              # Register Index
        w = x & ((1 << (32 - self.p)) - 1)  # Remainder for Zero Counting
        rho = self._get_rho(w)

        if rho > self.registers[j]:
            self.registers[j] = rho
```

The `add` function hashes the item, splits the bits into bucket index `j` and remainder `w`, counts trailing zeros in `w`, and updates the register.

Implementation: Count & Merge

```
    def count(self):
        # Checklist Item 3: Raw Estimate
        Z_inv = sum(2.0 ** -val for val in self.registers)
        E = self.alpha * (self.m ** 2) / Z_inv

        # Checklist Item 4: Range Corrections
        if E <= 2.5 * self.m: # Small Range (Linear Counting)
            V = self.registers.count(0)
            if V != 0:
                E = self.m * math.log(self.m / V)
        elif E > (1/30.0) * (1 << 32): # Large Range
            E = -(1 << 32) * math.log(1 - E / (1 << 32))
        return int(E)

    def merge(self, other):
        # Extra: Union Logic
        if self.p != other.p: raise ValueError("Precision mismatch")
        for i in range(self.m):
            if other.registers[i] > self.registers[i]:
                self.registers[i] = other.registers[i]
```

The Idea of Buckets

We split the stream into $m = 2^p$ substreams.

> Input Hash: 1010 00101...1010 (First 4 bits) $\rightarrow$ Bucket Index 10

Registers Array M:

An array of m integers (typically 5-bit integers) that stores the max rank for each bucket.

Why Hashing?

The "Meat Grinder"

A hash function (MurmurHash3) transforms arbitrary data ("Alice") into a fixed-size binary string.

The Avalanche Effect: Changing 1 bit of input flips ~50% of output bits. This ensures **Uniformity**.

> P(bit = 0) = 0.5
> P(bit = 1) = 0.5

```python
def murmur3_32(key, seed=0):
    """
    Pure Python implementation of MurmurHash3 (32-bit).
    Ensures consistent, well-distributed hashing without external dependencies.
    """
    key = bytearray(key.encode('utf-8')) if isinstance(key, str) else bytearray(k
    length = len(key)
    n_blocks = length // 4

    h1 = seed
    c1 = 0xcc9e2d51
    c2 = 0x1b873593

    for i in range(n_blocks):
        k1 = key[i*4] | (key[i*4+1] << 8) | (key[i*4+2] << 16) | (key[i*4+3] << 2
        k1 = (k1 * c1) & 0xffffffff
        k1 = ((k1 << 15) | (k1 >> 17)) & 0xffffffff
        k1 = (k1 * c2) & 0xffffffff
        h1 = (h1 ^ k1)
        h1 = ((h1 << 13) | (h1 >> 19)) & 0xffffffff
        h1 = (h1 * 5 + 0xe6546b64) & 0xffffffff

    tail_index = n_blocks * 4
    k1 = 0
```

Probabilistic Model: Bernoulli & Geometric

Intuition: Bernoulli Trials

Imagine flipping a coin. **Heads = 0**, **Tails = 1**.

Sequence: **0, 0, 0, 1** (3 Heads then Tails).

This is a "Bernoulli Process". If you see 10 Heads in a row, you know it's a rare event that implies many trials occurred.

> Rare Event ⇒ Large Count

Math: Geometric Distribution

Since bits are uniform (P=0.5), the position of the first 1-bit (ρ) follows a Geometric Distribution.

$$P(\rho = k) = \frac{1}{2^k}$$

Max Rank $M \approx log_2 n$

Stochastic Averaging

1 Register

Variance is Huge.

One "lucky" hash (e.g., hash("Zack") = 00000...) destroys the estimate.

m Registers

Variance is Low.

By averaging m independent experiments, lucky and unlucky buckets cancel out.

Derivation 1: Variance Reduction (Setup)

Step 1: The Independent Buckets

We split the data stream into m independent buckets. This creates m random variables $X_1, \dots, X_m$.

Let's assume each register X_i has a variance of σ^2.

Step 2: Variance of Sum

For independent variables, the variance of the sum is the sum of the variances.

$$\text{Var}\left(\sum_{i=1}^{m} X_i\right) = \sum_{i=1}^{m} \text{Var}(X_i) = m\sigma^2$$

(Variances add up linearly)

Derivation 1: Variance Reduction (Result)

Step 3: Variance of Mean

The estimator uses the Mean (Average) of the registers: $\bar{X} = \frac{1}{m} \sum X_i$.

Rule: When you scale a random variable by a constant C (here $1 / m$), the Variance scales by C^2.

$$\text{Var}(\bar{X}) = \text{Var}\frac{1}{m} \sum X_i = \frac{1}{m^2} (m\sigma^2) = \frac{\sigma^2}{m}$$

Step 4: Standard Error

$$\text{StdErr} = \text{Var}(\bar{X}) = \frac{\sigma^2}{m} = \frac{\sigma}{m}$$

The Indicator Z

The core of the HLL estimator is the **Harmonic Mean** of the register estimates.

$$Z = \sum_{j=1}^{m} 2^{-M[j]^{-1}}$$

Why Harmonic Mean?

Outlier Suppression:

Arithmetic Mean is pulled up by large values (outliers).

Harmonic Mean is dominated by small values. It effectively ignores the "lucky" buckets that saw 20 zeros and trusts the "crowd" of normal buckets.

Derivation: The Setup

Goal: Prove that $E_n[Z]$ relates to the cardinality n.

We assume a fixed cardinality n distributed into m buckets. This follows a **Multinomial Distribution**.

We need to sum over all possible ways to split n items into bucket sizes $n_1, n_2, \dots, n_m$.

The Constraint

$$n_1 + n_2 + \cdots + n_m = n$$

The buckets are dependent (coupled).

Derivation: Permutations

The "Candy Jar" Logic

1. Total Possibilities (m^n):
Each of the n items can go into any of the m buckets.
Total scenarios = $m \times m \ldots = m^n$.

2. Ways to Arrange:
The number of ways to divide distinct items into groups of specific sizes is the Multinomial Coefficient.

$$\frac{1}{m^n} \binom{n}{n_1, n_2, \ldots, n_m}$$

Derivation: Max Rank Probability

The y Term

For a bucket with n_j items, what is the probability the Max Rank is exactly k?

1. Prob that one item $\leq k$ is $(1 - 2^{-k})$.

2. Prob that **all** n_j items $\leq k$ is $(1 - 2^{-k})^{n_j}$.

3. Thus, Prob(Max = k):

$$y_{n_j, k} = \left(1 - \frac{1}{2^k}\right)^{n_j} - \left(1 - \frac{1}{2^{k-1}}\right)^{n_j}$$

The Final Expectation (Eq 4 in Paper):

$$E_n(Z) = \sum \frac{1}{\sum 2^{-k}} \sum_{\text{splits}} \frac{n!}{n_1! \dots} \frac{1}{m^n} \prod y_{n_j, k_j}$$

(This formidable sum is solved via Poissonization)

Derivation: Poissonization

The Decoupling Trick

The sum for fixed n is too hard because buckets are dependent ($n_1 + \cdots = n$).

Step 1: Poisson Model ($P(\lambda)$)

Assume N varies like a Poisson distribution. Buckets become independent with rate λ / m.

Step 2: Integral Identity

$$\frac{1}{Z} = \int_0^\infty e^{-Zt}\, dt$$

The Resulting Integral

Using this identity, the expected value becomes a product of integrals over independent buckets:

$$E_{P(\lambda)}(Z) = \int_0^\infty \sum_{k \geq 1} g_{\frac{x}{2^k}} e^{-t/2^k}\, m\, dt$$

This separates the variables, making the math solvable.

Derivation: The Integrals J_s

Defining the Constants

Using Mellin Transforms on the Poisson integral, the paper derives a family of integrals $J_s(m)$ that define the algorithm's behavior.

$$J_s(m) = \int_0^\infty u^s \log_2 \frac{2+u}{1+u} m \, du$$

Mapping to Constants

1. Bias Correction α_m:

$$\alpha_m = \frac{1}{mJ_0(m)}$$

2. Standard Error β_m:

$$\beta_m = m \frac{J_1(m)}{J_0(m)^2} - 1$$

Derivation: Laplace Method

Solving for Large m

We cannot solve $J_s(m)$ exactly. We use the **Laplace Method** for asymptotic expansion as $m \to \infty$.

The function mass concentrates near $u = 0$. We approximate:

$$f(u) \approx 1 - \frac{u}{2\ln 2}$$

The Final Expansions

$$J_0(m) \approx \frac{2\ln 2}{m} 1 + \frac{3\ln 2 - 1}{m}$$

Plugging this into the β_m formula yields:

$$\beta_\infty = 3\ln 2 - 1 \approx 1.04$$

Derivation: Depoissonization

The Return to Reality

We calculated expectation for random N (Poisson). We need it for fixed n.

Analytic Depoissonization:

Since the Poisson distribution is sharply peaked around $\lambda = n$, the paper proves mathematically that:

$$E_n(Z) = E_{P(n)}(Z) + O(1)$$

The error introduced by the Poisson trick is negligible for large n.

Thank You

Questions?